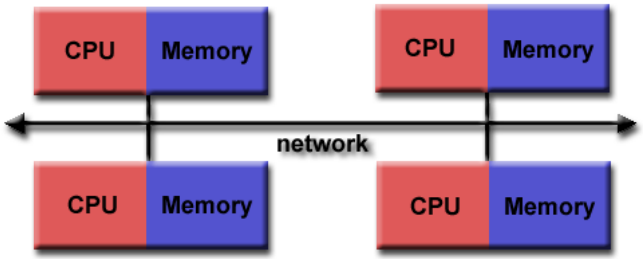
# 4160 – Distributed and Parallel Computing

P6 – MPI

CUHK

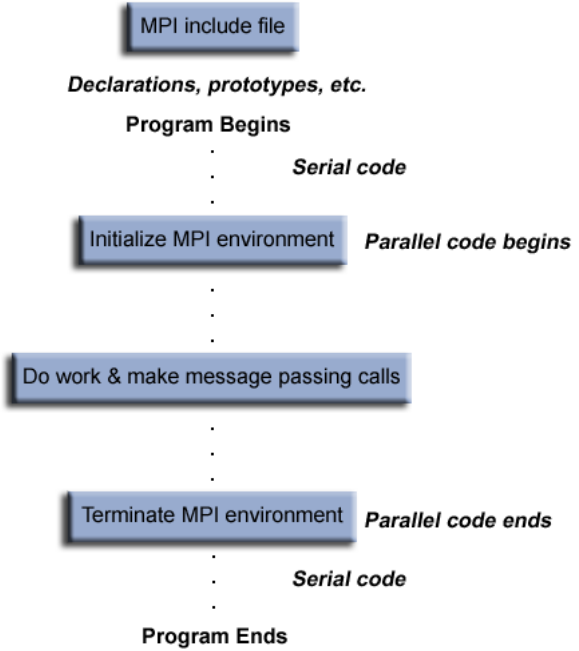
# Message Passing Interface

- Programming Model
  - Distributed Memory
    - i.e., no shared variable
  - Collaboration by message passing
  - Single Program Multiple Data





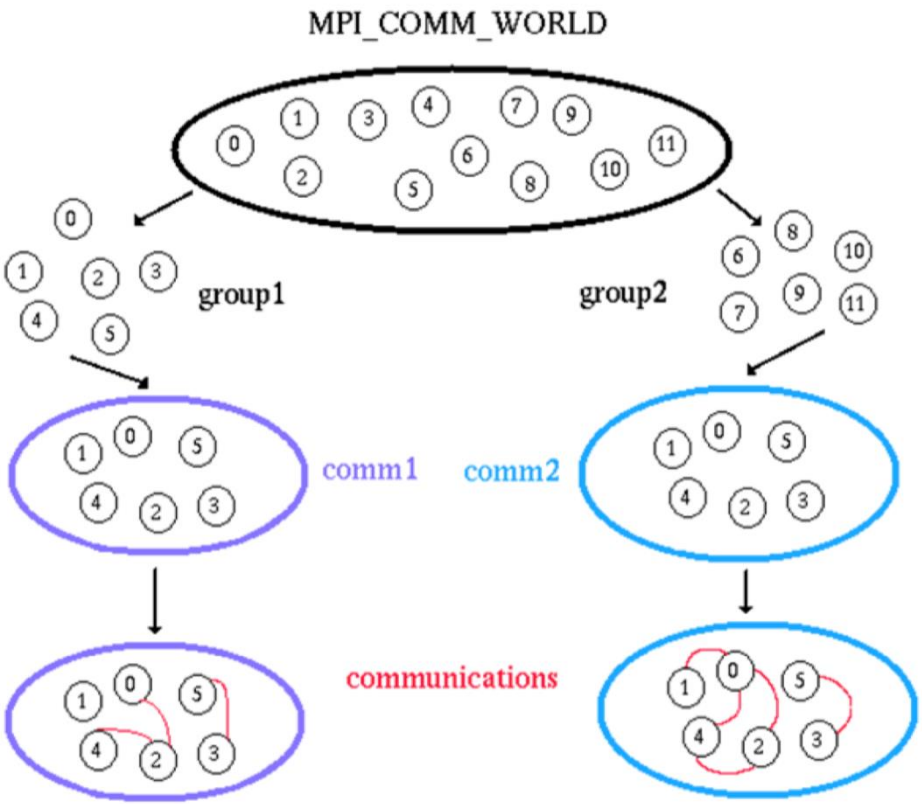
- Implementation (Library): MPICH, OpenMPI, …

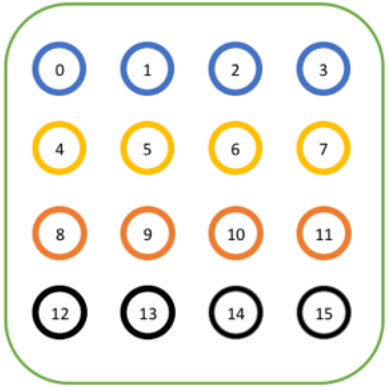


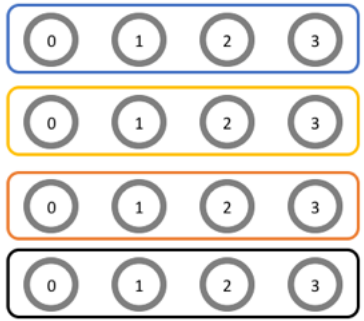

```
/>salloc –N 3 mpirun ./mpi_program_1
```

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    // Initialize the MPI environment
    MPI_Init(NULL, NULL);

    // Get the number of processes
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // Get the rank of the process
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

    // Get the name of the processor
    char processor_name[MPI_MAX_PROCESSOR_NAME];
    int name_len;
    MPI_Get_processor_name(processor_name, &name_len);

    // Print off a hello world message
    printf("Hello world from processor %s, rank %d out of %d processor
s\n",
           processor_name, world_rank, world_size);

    // Finalize the MPI environment.
    MPI_Finalize();
}
```

# Communicator and Rank




- **Rank = Node_ID**

# Point to Point communication

- There are different types of send and receive routines used for different purposes. For example:
  - Blocking send, Blocking receive, Non-blocking send (Isend()), Non-blocking receive, Buffered send, …
- Like Telephone communication
  - I call you, you need to pick up
  - A send(); B receive();
- **Any type of send routine can be paired with any type of receive routine**

```
// Find out rank, size
int world_rank;
MPI_Comm_rank(MPI_COMM_WORLD, &w
int world_size;
MPI_Comm_size(MPI_COMM_WORLD, &w

int number;
if (world_rank == 0) {
    number = -1;
    MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
} else if (world_rank == 1) {
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
    printf("Process 1 received number %d from process 0\n",
           number);
}
```

Get my world_rank

```
MPI_Send(
    void* data,
    int count,
    MPI_Datatype datatype,
    int destination,
    int tag,
    MPI_Comm communicator)
```

```
MPI_Recv(
    void* data,
    int count,
    MPI_Datatype datatype,
    int source,
    int tag,
    MPI_Comm communicator,
    MPI_Status* status)
```

```
MPI_Send(
    void* data,
    int count,
    MPI_Datatype datatype,
    int destination,
    int tag,
    MPI_Comm communicator)
```

//customizable message type

```
if (world_rank == 0) {
    number = -1;
    MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
} else if (world_rank == 1) {
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
    printf("Process 1 received number %d from process 0\n",
           number);
}
```

# Blocking vs Non-blocking

Blocking

- Send: When the control is returned, the message has been sent and everything that has to do with the send is finished
- Receive: When control is returned, the whole message has been received

Non-blocking:

- Send: The control is returned immediately, the actual send is done later by the system
- Receive: The control is returned immediately, the message is not received until arrives; if you really access the message content b4 it is actually arrived, you are at your own risk (you can use MPI_test periodically to check if the content is ready)

You may mix a blocking send with a non-blocking receive

# Non-blocking: overlapping but harder programming

- Non-blocking, as usual, can help overlapping
  - Isend()  //return immediately
  - check-if-sending done every 10s, if not {

    Do some other useful work…

    }
- Overlapping compute with communication

# Example: Ring program

```
int token;
if (world_rank != 0) {
    MPI_Recv(&token, 1, MPI_INT, world_rank - 1, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process %d received token %d from process %d\n",
           world_rank, token, world_rank - 1);
} else {
    // Set the token's value if you are process 0
    token = -1;
}
MPI_Send(&token, 1, MPI_INT, (world_rank + 1) % world_size,
         0, MPI_COMM_WORLD);

// Now process 0 can receive from the last process.
if (world_rank == 0) {
    MPI_Recv(&token, 1, MPI_INT, world_size - 1, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process %d received token %d from process %d\n",
           world_rank, token, world_size - 1);
}
```

# Message Ordering

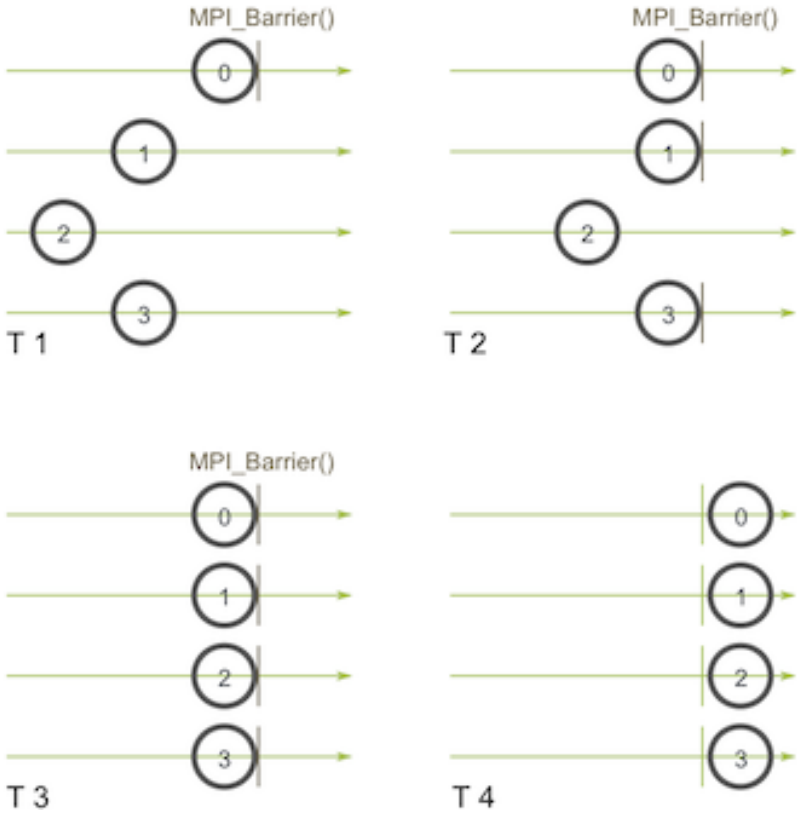
- **Message In-Order is enforced in MPI design:**
  - Despite the network routing delay possibility, if a sender process sends two messages (Message 1 and Message 2) in succession to the same destination process, the receive operation will receive Message 1 before Message 2.
  - Order rules do not apply if there are multiple threads participating in the communication operations.

# Collective communication

- MPI_Bcast
- MPI_Scatter & MPI_Gather
- MPI_Allgather
- MPI_Reduce and MPI_AllReduce

# Collective communication

- Involves participation of **all** processes in a communicator
- That is, a collective communication won't further proceed until all processes are in-sync
- In other words, collective communication implies an insertion of MPI_**Barrier** among processes
    - All processes must reach a point in their code before they can all begin executing again.



# MPI_Barrier

Blocks until all processes in the communicator have reached this routine.

## Synopsis

```
int MPI_Barrier(MPI_Comm comm)
```

## Input Parameters

**comm**
: communicator (handle)

## Notes

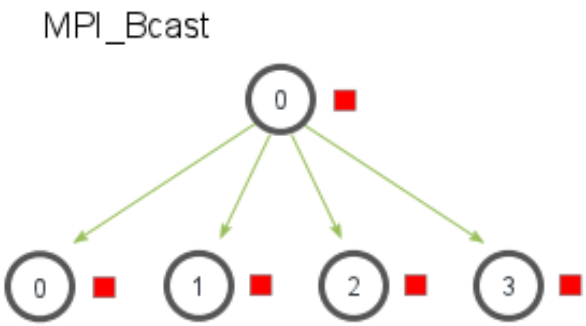
Blocks the caller until all processes in the communicator have called it; that is, the call returns at any process only after all members of the communicator have entered the call.

- Everyone calls MPI_Barrier()
- MPI_Barrier() returns only after everyone has called it

# How to implement MPI_barrier?

- One way is to use our ‘ring-program’
  - The first one reaches the barrier passes a token the its neighbor
  - Until the first one receives back the token
  - Then do a broadcast to tell everybody to proceed

# MPI_Bcast

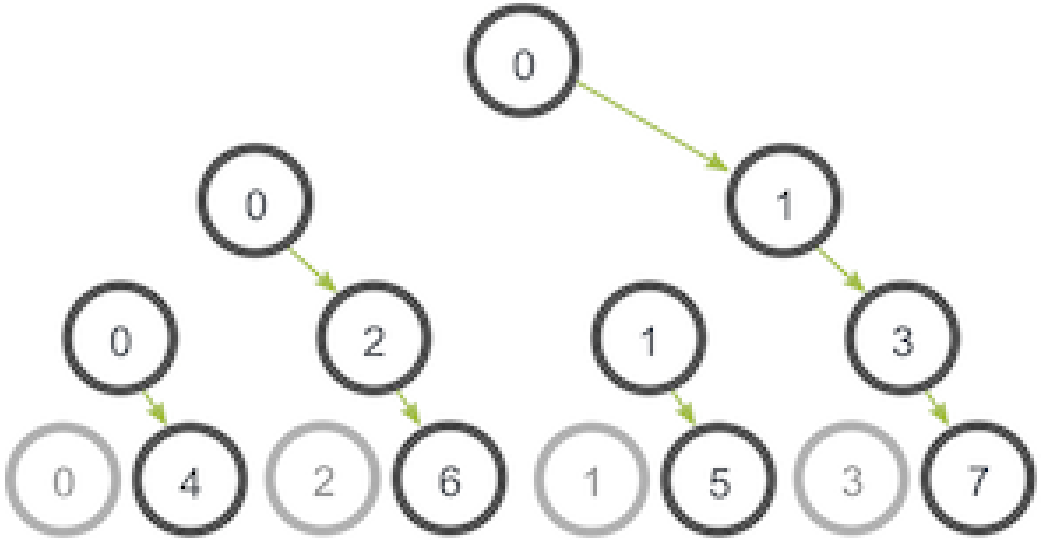



- A baseline implementation
  - *network level multicast() is seldom used in application level
  - *pseudo-code excludes sending to itself

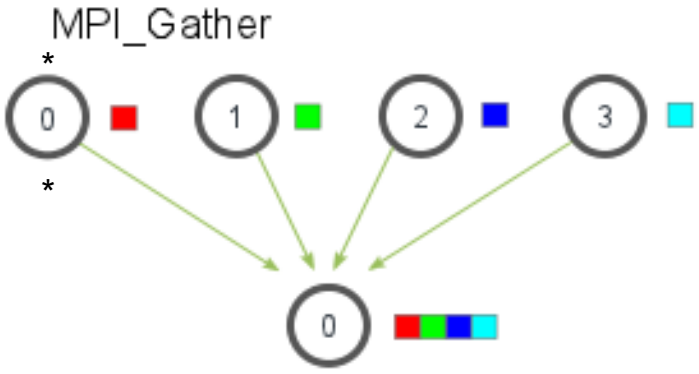
```
void my_bcast(void* data, int count, MPI_Datatype datatype, int root,
              MPI_Comm communicator) {
  int world_rank;
  MPI_Comm_rank(communicator, &world_rank);
  int world_size;
  MPI_Comm_size(communicator, &world_size);

  if (world_rank == root) {
    // If we are the root process, send our data to everyone
    int i;
    for (i = 0; i < world_size; i++) {
      if (i != world_rank) {
        MPI_Send(data, count, datatype, i, 0, communicator);
      }
    }
  } else {
    // If we are a receiver process, receive the data from the root
    MPI_Recv(data, count, datatype, root, 0, communicator,
             MPI_STATUS_IGNORE);
  }
}
```

# MPI_Bcast

- More efficient implementation
  - Tree-based



# MPI_Gather




- Collect elements from each process to the designated 'root' process
  - E.g., root = 0;
  - E.g., root = 2;
- Gathered elements are ordered based on rank of the processes

```
MPI_Gather(
    void* send_data,
    int send_count,
    MPI_Datatype send_datatype,
    void* recv_data,
    int recv_count,
    MPI_Datatype recv_datatype,
    int root,
    MPI_Comm communicator)
```

```
MPI_Comm comm;
    int gsize,sendarray[100];
    int root, myrank, *rbuf;
    ...
    MPI_Comm_rank( comm, myrank);
    if ( myrank == root) {
       MPI_Comm_size( comm, &gsize);
       rbuf = (int *)malloc(gsize*100*sizeof(int));
       }
    MPI_Gather( sendarray, 100, MPI_INT, rbuf, 100, MPI_INT, root, comm);
```

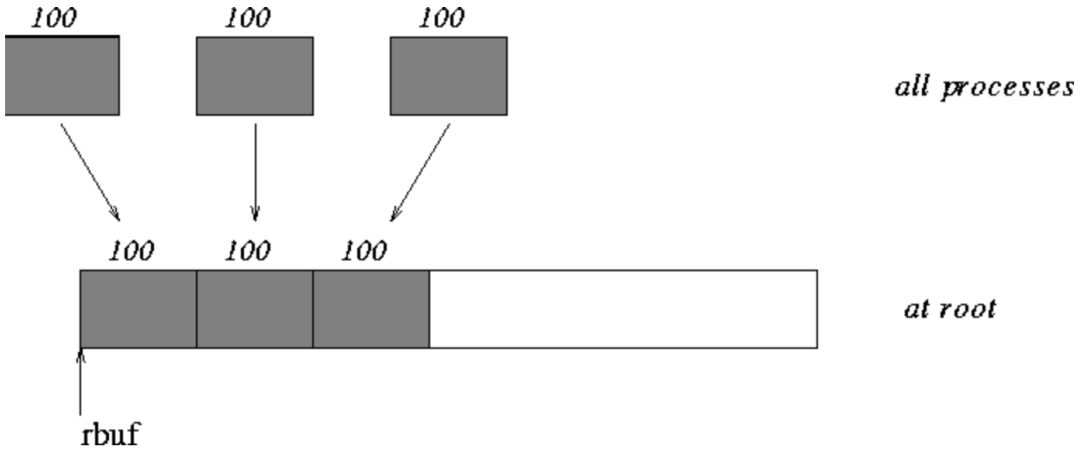


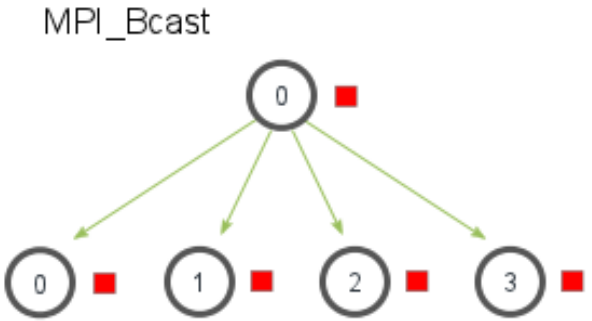

**Figure 3:** The root process gathers 100 `ints` from each process in the group.

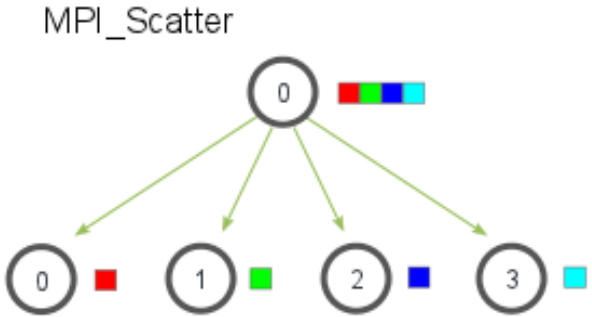
- Anyone.MPI_Gather()
  - Send my sendarray to root
  - MPI_Barrier()
- Root.MPI_Gather()
  - Receive from all and write to rbuf
  - MPI_Barrier()

Ref: MPI_forum

# MPI_Scatter

- MPI_Bcast sends the *same* piece of data to all processes vs.
- MPI_Scatter sends different *chunks of an array* to different processes




```
MPI_Scatter(
    void* send_data,
    int send_count,
    MPI_Datatype send_datatype,
    void* recv_data,
    int recv_count,
    MPI_Datatype recv_datatype,
    int root,
    MPI_Comm communicator)
```

If send_count = 2;
process 0 in the communicator get send_data[0-1]
process 1 in the communicator get send_data[2-3]

If send_count = 4;
process 0 in the communicator get send_data[0-3]
…

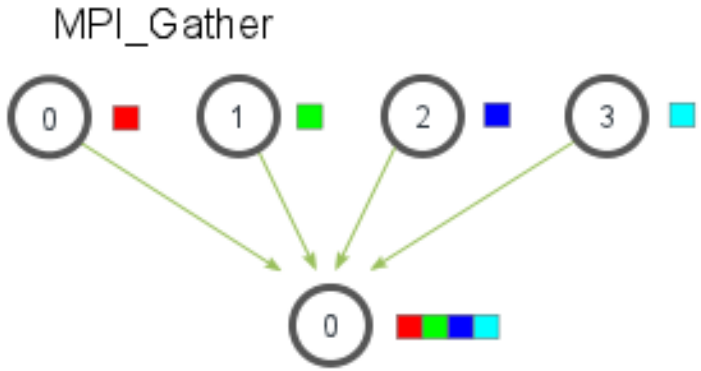
# Example: Distributed average of an array of random numbers

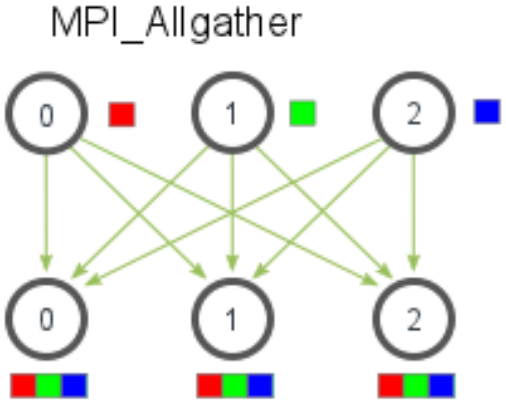
- Scatter the numbers to all processes, giving each process an equal amount of numbers.
- Each process computes the average of their subset of the numbers.
- Gather all averages to the root process. The root process then computes the average of these numbers to get the final average.

```
if (world_rank == 0) {
  rand_nums = create_rand_nums(elements_per_proc * world_size);
}

// Create a buffer that will hold a subset of the random numbers
float *sub_rand_nums = malloc(sizeof(float) * elements_per_proc);

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
MPI_Gather(&sub_avg, 1, MPI_FLOAT, sub_avgs, 1, MPI_FLOAT, 0,
           MPI_COMM_WORLD);

// Compute the total average of all numbers.
if (world_rank == 0) {
  float avg = compute_avg(sub_avgs, world_size);
}
```

# MPI_Allgather

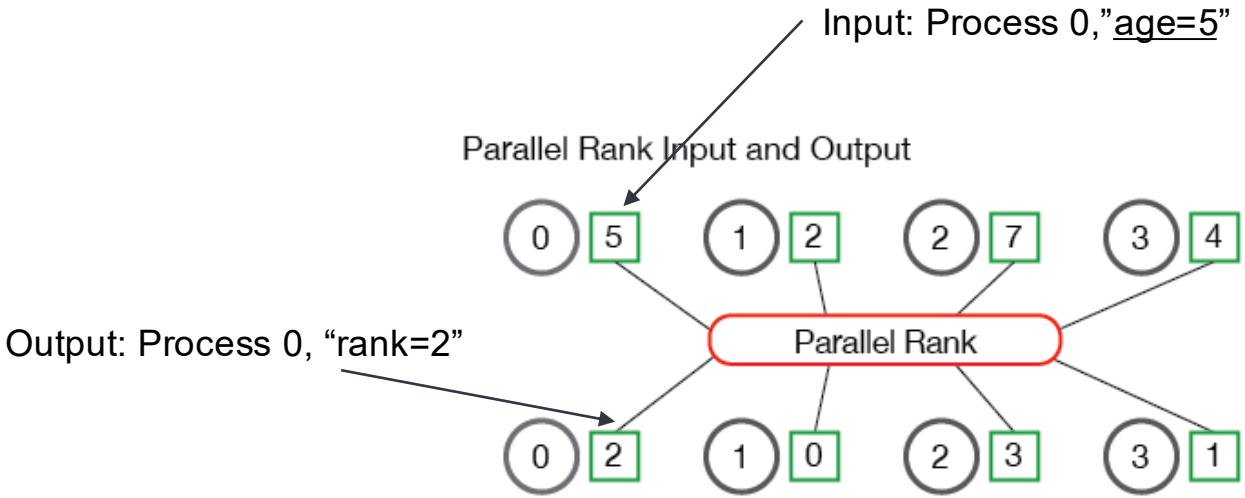
- Many-to-Many
- Gather all elements to *all* processes
  - = Gather + Bcast

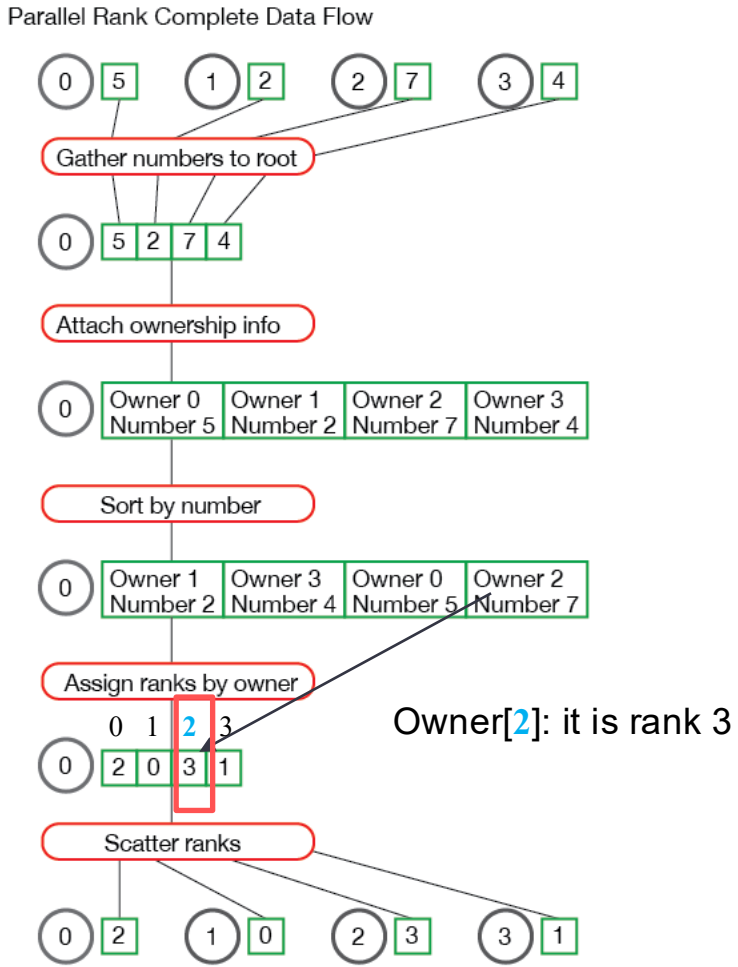


# Example: Parallel Rank Problem

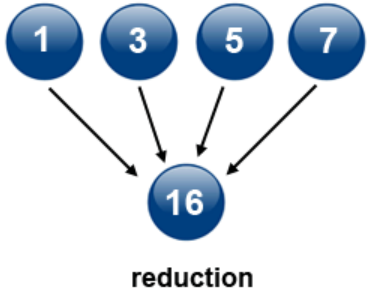
- Example Usage: leader election (as MPI is for reliable cluster; easy)
  - 4 computers
  - Rank by ‘age’

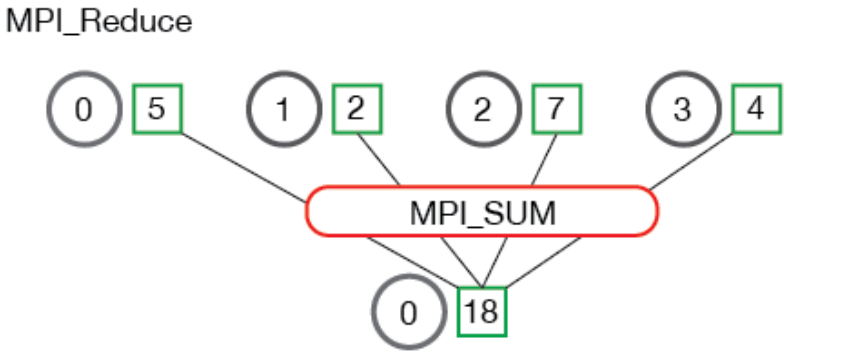


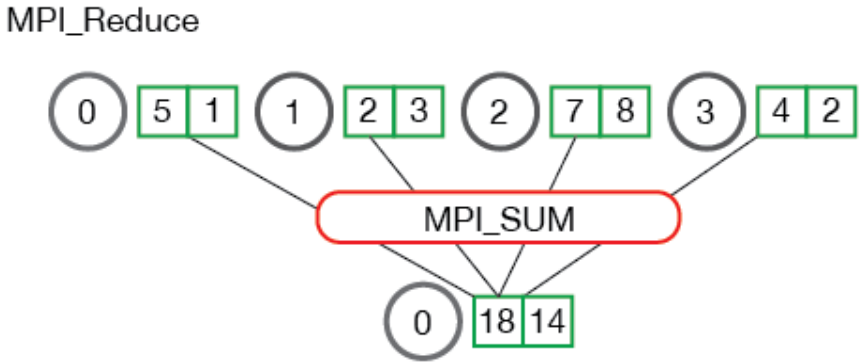



Owner[2]: it is rank 3

# MPI_Reduce


1
3
5
7
16
reduction


MPI_Reduce
0
5
1
2
2
7
3
4
MPI_SUM
0
18


MPI_Reduce
0
5
1
1
2
3
2
7
8
3
4
2
MPI_SUM
0
18
14

# Example Revisit: Average using Reduce

```c
float *rand_nums = NULL;
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
int i;
for (i = 0; i < num_elements_per_proc; i++) {
  local_sum += rand_nums[i];
}

// Print the random numbers on each process
printf("Local sum for process %d - %f, avg = %f\n",
       world_rank, local_sum, local_sum / num_elements_per_proc);

// Reduce all of the local sums into the global sum
float global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM, 0,
           MPI_COMM_WORLD);

// Print the result
if (world_rank == 0) {
  printf("Total sum = %f, avg = %f\n", global_sum,
         global_sum / (world_size * num_elements_per_proc));
}
```

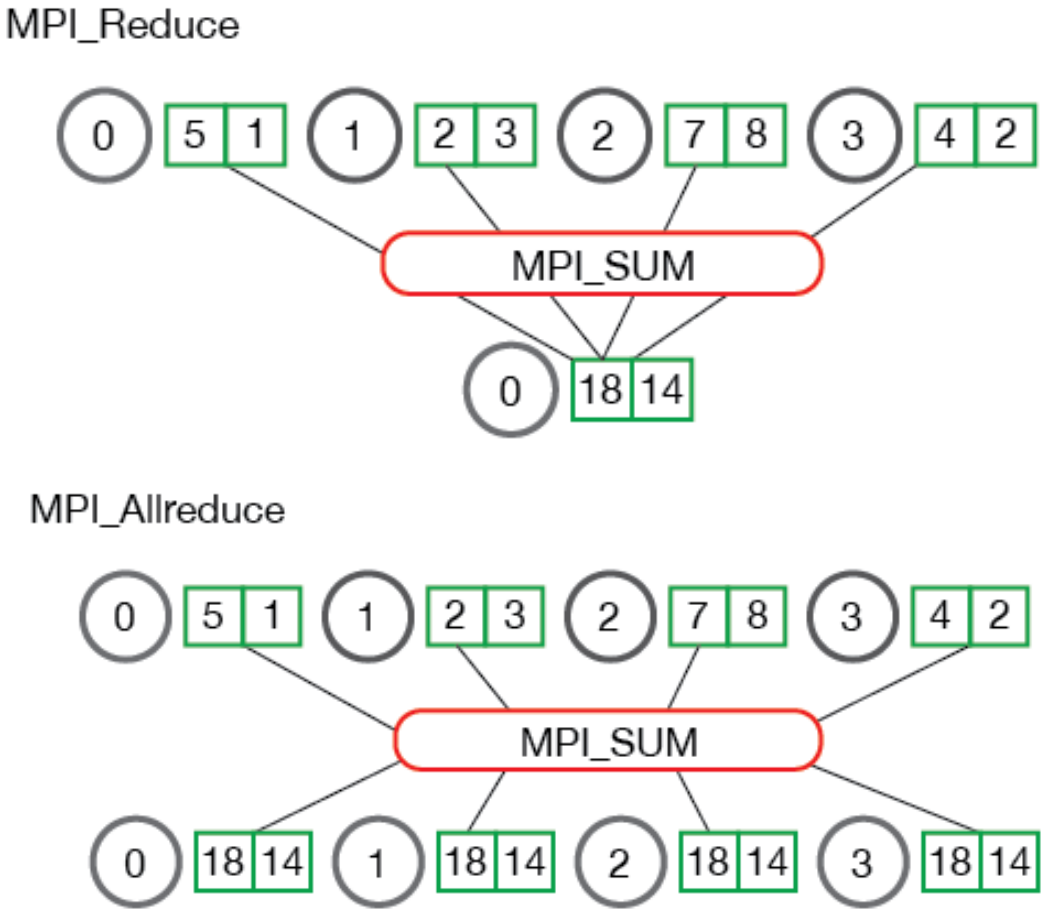
- Each process creates random numbers and make a local_sum calculation
- Each process computes their local sums
- Reduce the local_sums to global_sum

vs

```c
if (world_rank == 0) {
  rand_nums = create_rand_nums(elements_per_proc * world_size);
}

// Create a buffer that will hold a subset of the random numbers
float *sub_rand_nums = malloc(sizeof(float) * elements_per_proc);

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
  sub_avgs = malloc(sizeof(float) * world_size);
}
MPI_Gather(&sub_avg, 1, MPI_FLOAT, sub_avgs, 1, MPI_FLOAT, 0,
           MPI_COMM_WORLD);

// Compute the total average of all numbers.
if (world_rank == 0) {
  float avg = compute_avg(sub_avgs, world_size);
}
```

# MPI_AllReduce

- =Reduce + Bcast



# MPI also supports threading

- MPI_THREAD_SINGLE - Level 0: Only one thread will execute.
- MPI_THREAD_FUNNELED - Level 1: The process may be multi-threaded, but only the main thread will make MPI calls - all MPI calls are funnelled to the main thread.
- MPI_THREAD_SERIALIZED - Level 2: The process may be multi-threaded, and multiple threads may make MPI calls, but only one at a time. That is, calls are not made concurrently from two distinct threads as all MPI calls are serialized.
- MPI_THREAD_MULTIPLE - Level 3: Multiple threads may call MPI with no restrictions.

# Load Balance Revisit: List all prime numbers < N

- **Sieve of Eratosthenes algorithm:**

Step 1. Generate a list for $2, 3, 4, \cdots$, and $N$;
Step 2. Let $k = 2$, the first prime in the list;
Step 3. Repeat the following procedure:

- Delete all multiples of $k$ in the region $[k^2, N]$.
- Locate the smallest number $> k$. Set the new $k$ to it.
- Until $k^2 > N$.

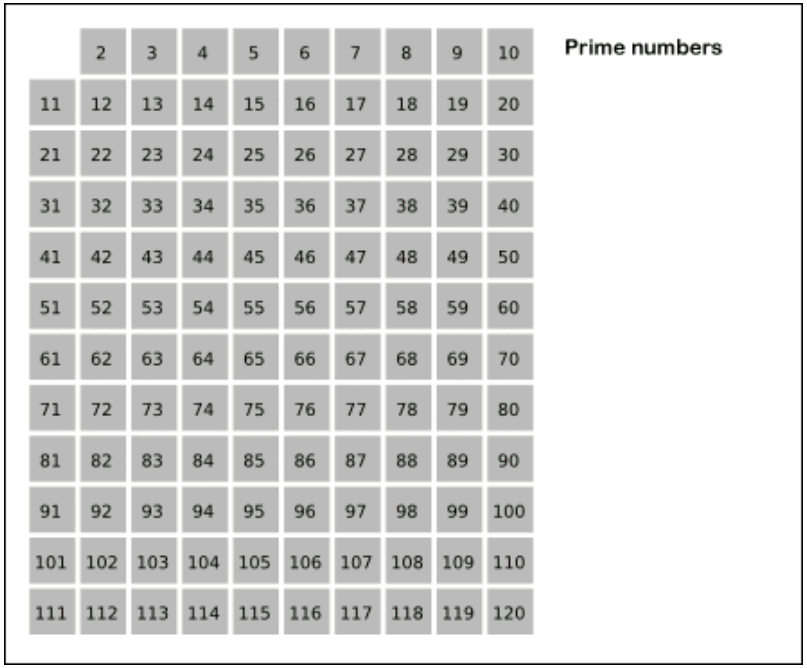
Step 4. All remaining numbers are primes.



Ref: Guan@LSU

# Ex. Find all primes

- https://en.wikipedia.org/wiki/Eratosthenes#Prime_numbers




| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |
| 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 |
| 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 |
| 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 |
| 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 |
| 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 |
| 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 |
| 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 |
| 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 |
| 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120 |

# Make an MPI version

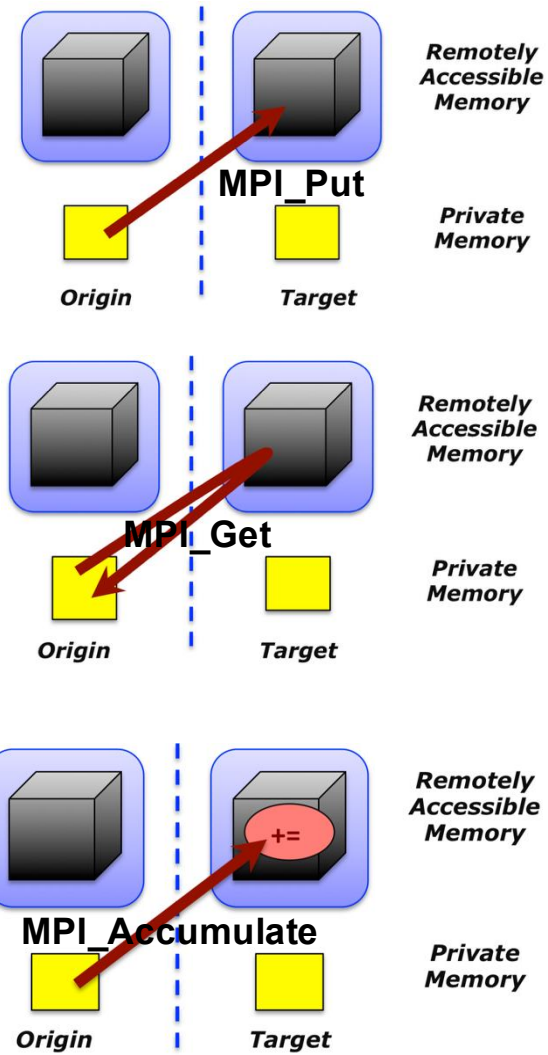
- Example Challenge: how to decompose the problem?
  - E.g., # of processes=4, so:
    - Process p1: 2, 6, 10, 14, 18, …
    - Process p2: 3, 7, 11, 15, 19, …
    - Process p3: 4, 8, 12, 16, 20, …
    - Process p4: 5, 9, 13, 17, 21, …
  - What's the problem?
    - Ans: after the first round with k=2
      - p1 and p3 have eliminated all their local numbers and nothing to do!

# Communication

- Two-sided communication
  - Like a Phone call
    - I call you (send), you must need to pick up (recv)
- One-sided communication
  - Like the target (you) open a shared dropbox (called window; a subset of your local memory) for an extended period
  - Let the origin (me / many others) to put and read the content there freely during that period
  - You don’t need to have a for-loop of recv

# One-sided communication (RMA)

- Basic primitives:
  - **MPI_Win_create** and **MPI_Win_free**
    - For the target (destination) to create or remove a window
  - **MPI_Put**
    - For the origin to put data from local memory to remote memory
  - **MPI_Get**
    - For the origin to get data from remote memory to local memory
  - **MPI_Accumulate(MPI_SUM or MPI_PROD…)**
    - For the origin to update remote memory
- Data movements are non-blocking



# Deployment

- Implementation Library: Open MPI and MVAPICH
- Open MPI would automatically choose the highest-bandwidth port for us
  - RDMA vs Ethernet
  - The nice programming abstraction of MPI library inevitably incur overheads
  - So in one extreme, highest-performance systems would directly use RDMA verbs, like writing assembly directly
- Otherwise, just `mpirun` is fine

# Ethernet:

- Kernel is involved in low-level networking (control plane + data plan)

| A | B |
|---|---|
| `send(msg)` //OS syscall, networking stack… | |
| | `msg = receive()` //OS syscall, networking stack … |
| | OS copies `msg` from NiC buffer to kernel space and then to user space |

# 2-sided RDMA

- Kernel (OS) bypass on data copying from rNIC to user space

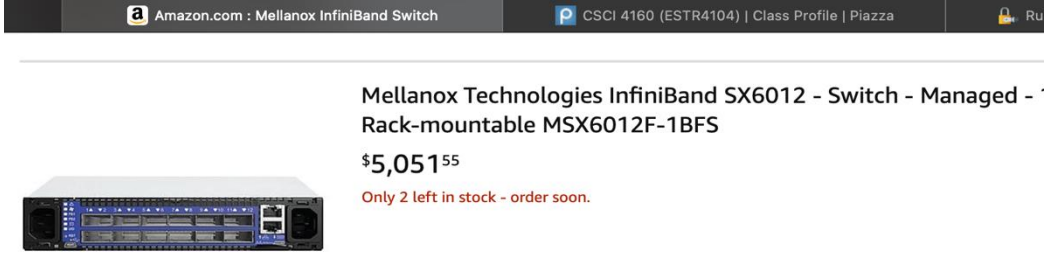
| A | B |
| --- | --- |
| MMap user-space memory to RDMA device | MMap user-space memory for RDMA device |
| `RMDA_send(msg) //will also write a doorbell to RDMA register to notify rNIC about this; rNIC will then do the sending` | |
| | `Forloop:`<br>`  msg = RMDA_receive()`<br>`//CPU tells rNIC to receive`<br>`//rNIC will copy received msg to user space straight` |

# 1-sided RDMA

- Kernel (OS) + remote CPU data plane bypass

| A | B |
|---|---|
| MMap user-space memory to RDMA device | - MMap user-space memory to RDMA device |
| - Request k B for direct RDMA access | - Authorized A for direct RDMA access |
| `RMDA_write(msg, 0x1234) //will also write a doorbell to RDMA register to notify rNIC about this; rNIC will then do the sending` | |
| ...<br>Keep writing<br>... | ~~`//CPU tells rNIC to receive`~~<br>`//rNIC copies msg to user space buffer` |
| `Use RMDA_send / RMDA_write_with_immediate to notify the end of this round of writing` | `//get notified about new data arrival` |

# RDMA products




- Nvidia (Acquired Mellanox)
  - Software:
    - Now call as GPUDirect (RDMA)
  - Two types of Network protocol and hardware
    - InfiniBand IB (everything is dedicated for RDMA)
      - More expensive but reliable
    - RoCE (build on top on Ethernet switches)
      - More economic but a bit weaker than IB

# References


- https://computing.llnl.gov/tutorials/mpi/
- https://mpitutorial.com/tutorials/
- https://bt.nitk.ac.in/c/17a/co471/notes/3.MPI.pdf
- https://www.hlrn.de/twiki/pub/NewsCenter/ParProgWorkshopFall2017/ws_mpi3_onesided.pdf
- http://wgropp.cs.illinois.edu/courses/cs598-s16/lectures/lecture34.pdf
- http://pages.tacc.utexas.edu/~eijkhout/pcse/html/mpi-onesided.html